Assignment Title: Building an Enhanced Custom Shell

I. Objectives

Build a functional, interactive UNIX-like custom shell program in C that can perform standard command line interactions, execute system commands using process management APIs, and directly invoke POSIX system calls for system and file administration.

a. Accept interactive user commands from standard input with a prompt.
b. Parse and tokenize user input strings into command tokens and arguments.
c. Fork child processes and execute programs using the exec family of system calls.
d. Implement essential built-in commands and POSIX utility system calls directly.

II. Implement Shell Commands & System Calls

Your shell program must directly implement or execute the following set of commands. To demonstrate a deep understanding of OS principles, you MUST NOT use system() or invoke standard system binaries for manual utilities. Instead, use proper POSIX system calls (e.g., fork(), execvp(), open(), read(), write(), stat(), chdir(), getcwd(), mkdir(), rmdir(), unlink(), waitpid()).

1. Standard File & Directory Inspection Commands

- **pwd –** Print current working directory (using getcwd system call).
- **ls –** List directory contents (using opendir, readdir, closedir, and stat).
- **cd –** Change working directory (builtin using chdir system call).
- **whoami –** Display current user identity (using getuid and getpwuid).
- **uname –** Display system name and architecture details (using uname system call).

2. File & Directory Manipulation Commands

- **mkdir –** Create new directory with mode permissions (using mkdir system call).
- **touch –** Create an empty file or update timestamp (using open with O_CREAT/utimensat).
- **rm –** Remove file (using unlink/remove system call) or empty directory (using rmdir).
- **cp –** Copy source file contents to destination file (using read, write system calls).
- **mv –** Move or rename file/directory (using rename system call).

3. File Viewing & Process Inspection Commands

- **cat –** Concatenate and display file content on stdout (using open, read, write).
- **echo –** Print text arguments to standard output (builtin).
- **uptime –** Display operating system uptime and CPU load stats (from /proc/uptime & /proc/loadavg).
- **meminfo –** Display memory statistics (parsing /proc/meminfo).

Important Implementation Rules:

- Do NOT rely on system shell execution (e.g., system("ls") is strictly prohibited).

- Use process control system calls: fork(), execvp()/execv(), wait()/waitpid().
- Use POSIX file handling APIs and system calls: open(), read(), write(), close(), stat(), opendir(), readdir().

III. Built-in Command Handling

The following commands MUST be handled natively inside the main parent process without spawning a child process (since they modify the shell state or require direct shell evaluation):

- **cd –** Change current directory of the shell process using chdir(). Support cd .. and cd with no args.
- **echo –** Print strings or shell environment variables (e.g., echo $PATH, echo Hello).
- **exit –** Terminate the shell program gracefully with an exit status code.
- **help –** Display built-in command help menu and general usage guidelines.

IV. Command Parsing & Execution Loop

a. Command Arguments: Support multiple command-line arguments (e.g., ls -l /usr/bin, cp src.txt dest.txt).

b. Error Handling: Detect and print clear error messages for invalid commands, missing arguments, or file access errors (using perror() or strerror()).

c. Interactive Loop: Run in a continuous interactive loop showing prompt (e.g., myshell>) until exit is entered.

V. Advanced Features

Implement any one or combination of the following advanced features for extra credit:

- **1. Command Piping (|):** Connect stdout of one command to stdin of another using pipe() and dup2().
- **2. I/O Redirection (>, <, >>):** Redirect standard input (<) and standard output (>) using open() and dup2().
- **3. Background Execution (&):** Execute commands asynchronously in background when appended with & (without blocking parent).
- **4. Command History:** Maintain an in-memory list of recent command history accessible via history command.